

## Theory and Methodology

# On the complexity of generalized due date scheduling problems

Nicholas G. Hall

*Faculty of Management Sciences, The Ohio State University, Columbus, OH, USA*

Suresh P. Sethi

*Faculty of Management, University of Toronto, Canada*

Chelliah Sriskandarajah \*

*GERAD, École Polytechnique, Montréal, Canada*

**Abstract:** We study the recently identified class of generalized due date scheduling problems. These are machine scheduling problems for which due dates are specified according to the position in which a job is completed, rather than the identity of that job. Flexible manufacturing environments and public sector planning problems provide applications. We study a wide variety of these problems, with a view to determining their computational complexity. In several instances a problem which is NP-hard under a traditional due date definition admits an efficient algorithm under the new definition. As well as determining the complexity of many generalized due date scheduling problems, including one published open problem, we also describe several problems, the complexity of which is still unresolved.

**Keywords:** Machine scheduling, complexity theory, due dates, NP-complete

### 1. Introduction and motivation

The traditional view of due dates, in the context of machine scheduling problems, is that we are given a due date  $d_j$ , by which time job  $j$  should be completed. Thus, each due date is *specific* to a particular job. An alternative perspective was provided by Hall (1986), where interchangeability of the jobs means that what matters is *how many* jobs have been completed by any point in time, rather than which jobs they are. The

use of these *generalized due dates* generates an entire new class of scheduling problems.

A simple example shows the difference which results from this new perspective. Under the traditional view of *specific due dates*, we may be given jobs  $A$ ,  $B$  and  $C$  and corresponding due dates  $d_A$ ,  $d_B$  and  $d_C$ . The scheduling problem is to complete job  $A$  by  $d_A$ , job  $B$  by  $d_B$ , and job  $C$  by  $d_C$ . Under the assumption of generalized due dates, we are given jobs  $A$ ,  $B$  and  $C$ , as well as due dates  $d_1$ ,  $d_2$  and  $d_3$ , where  $d_1 \leq d_2 \leq d_3$ . Then the scheduling objective is to complete at least one job by  $d_1$ , at least two jobs by  $d_2$ , and all three jobs by  $d_3$ .

Hall (1986) described a number of applications in which the generalized due dates definition is

\* Presently at Dept. of Industrial Engineering, University of Toronto, Canada.

Received December 1988; revised July 1989

appropriate. These applications include public-utility planning problems, survey design, and scheduling problems in some manufacturing environments. A particular instance occurs in the petrochemical industry, where a number of interchangeable heat exchangers must be maintained. The repair time of a heat exchanger is estimated with respect to the extent of corrosion and erosion on it. Since a number of heat exchangers must be serviced by a certain date, the problem can be formulated as a generalized due date scheduling problem. It may also be noted that the generalized due date assumption is less restrictive than, and therefore more flexible than, the specific due date assumption. For a discussion of the importance of flexibility in manufacturing, see, e.g., Stecke and Solberg (1981) and Browne et al. (1984).

In view of the numerous potential applications, and the richness in problem variety, of this new scheduling class, we investigate many different problems in this paper. More specifically, we define the generalized due date counterpart of most of the well-studied machine scheduling problems in the literature. The emphasis is on establishing each problem's computational complexity by proving equivalence to an existing hard problem, or by providing an efficient algorithm. We solve an open problem described by Hall (1986) and also provide several problems with still unresolved complexity.

The outline of the paper is as follows. In the next section, we introduce the notation. Section 3 deals with the generalized due date problem with a single machine. Multiple machine problems are treated in Section 4. Our results are summarized in Table 1. Table 2 presents a list of open problems. Section 5 concludes the paper.

## 2. Background and notation

For an introduction to machine scheduling problems, the reader is referred to Conway, Maxwell and Miller (1967), or Baker (1974). An introduction to computational complexity can be found in the works of Garey and Johnson (1979), and Papadimitriou and Steiglitz (1982). Lenstra, Rinnooy Kan and Brucker (1977) studied the complexity of a wide variety of scheduling problems using the specific due date definition. The scheduling problems are basically optimization

problems that can be stated as follows: consider the allocation of machines to activities known as jobs. In general,  $n$  jobs are processed on a certain group of  $m$  machines in a shop, so as to optimize certain objectives such as finish time, mean flow time, etc., subject to various constraints, e.g., on processing technology, or job characteristics. A shop consists of  $m \geq 1$  machines:  $\{M_i | 1 \leq i \leq m\}$ . There are  $n$  jobs:  $\{J_j | 1 \leq j \leq n\}$  which have to be scheduled on those machines such that a machine can process only one job at a time, and a job can be processed by only one machine at a time. We give a brief description of the shop models considered in the literature.

In single machine (denoted as 1) problems, each job  $J_j$  has only one task (with processing time  $p_j$ ) to be worked on the machine. In parallel machine (denoted as  $P$ ) problems, we have more than one machine and each job has only one task which can be processed on any machine. In a flowshop (denoted as  $F$ ), job  $J_j$  has  $m$  tasks,  $\{t_j^i | 1 \leq i \leq m\}$ , each with processing time  $p_j^i$  on machine  $M_i$ . Job  $J_j$  will first be processed on  $M_1$  for  $p_j^1$  units of time, and then on  $M_2$  for  $p_j^2$  units of time, and so on until it finally completes its processing on  $M_m$  for  $p_j^m$  units of time. In the case of an openshop (denoted as  $O$ ), each job has  $m$  tasks as in a flowshop, but the tasks may be processed in any order. However, in a jobshop (denoted as  $J$ ) this order may vary with the job and more than one task may be processed on the same machine. The terminology used in this paper follows the classification  $\alpha|\beta|\gamma$  of scheduling problems suggested by Graham et al. (1979), where  $\alpha$  indicates the machine environment, (e.g.,  $F2$  for a flowshop with 2 machines),  $\beta$  denotes the job characteristics or restrictive requirements (e.g., generalized due dates (gdd)), and  $\gamma$  is the optimality criterion (e.g., finish time, mean flow time, maximum lateness, etc.). We also make use of the following notation:

|           |                                                                                     |
|-----------|-------------------------------------------------------------------------------------|
| $w_j$     | = weight (value) of job $j$ ,                                                       |
| $w_{[j]}$ | = weight (value) of the $j$ -th job to be completed,                                |
| $r_j$     | = ready time of job $j$ ,                                                           |
| $C_j$     | = completion time of job $j$ ,                                                      |
| $C_{[j]}$ | = completion time of the $j$ -th job to be completed,                               |
| $d_{[j]}$ | = due date of the $j$ -th job to be completed for the generalized due date version, |

- $d_j$  = due date of job  $j$  for the specific due date version,  
 $T_{[j]}$  =  $\max\{0, C_{[j]} - d_{[j]}\}$ , the tardiness of the  $j$ -th job to be completed,  
 $U_{[j]} = \begin{cases} 1 & \text{if } C_{[j]} > d_{[j]}, \\ 0 & \text{otherwise,} \end{cases}$   
 $\bar{d}_{[j]}$  = deadline, by which time the  $j$ -th job must be completed.

We define the optimality criterion as the minimization of

- (a) the finish time of jobs,  $C_{\max} = \max_j \{C_j\}$ ,  
 (b) the total completion time of jobs,  $\sum C_j$ ,  
 (c) the maximum lateness of jobs,  $L_{\max} = \max_j \{C_{[j]} - d_{[j]}\}$ ,  
 (d) the number of late jobs,  $\sum U_{[j]}$ , where  $U_{[j]} = 0$  if  $C_{[j]} \leq d_{[j]}$ ,  $U_{[j]} = 1$  otherwise.  
 (e) the total tardiness of jobs,  $\sum T_{[j]}$ , where  $T_{[j]} = \max\{0, C_{[j]} - d_{[j]}\}$ .

There are no assumptions made in this paper about the schedule  $d_{[j]}$ ,  $j = 1, \dots, n$ , of generalized due dates, except the very natural assumption  $d_{[1]} \leq d_{[2]} \leq \dots \leq d_{[n]}$ . That is, the due date for the  $i$ -th job to be completed is no greater than for the  $j$ -th job to be completed, for  $1 \leq i < j < n$ .

### 3. Single machine problems

In this section we investigate the computational complexity of several single machine scheduling problems with generalized due dates. Whenever we describe a problem as NP-complete we refer of course to the recognition version of the problem. We begin by deciding the complexity of an open problem described by Hall (1986).

**Problem 1.**  $1|gdd, r_j|L_{\max}$ .

We show that Problem 1 is NP-complete in the strong sense by transformation from the 3-partition problem (Garey and Johnson, 1979, p. 224). Recall that 3-partition can be stated as: given positive integers  $N$ ,  $B$  and a set of integers  $A = \{a_1, a_2, \dots, a_{3N}\}$ , with  $\sum_{j=1}^{3N} a_j = NB$  and  $\frac{1}{4}B < a_j < \frac{1}{2}B$  for  $1 \leq j \leq 3N$ , does there exist a partition of  $A$  into sets  $A_1, A_2, \dots, A_N$ , each of cardinality 3, such that  $\sum_{j \in A_k} a_j = B$ ,  $k = 1, \dots, N$ ? Let  $N$ ,  $B$  and  $A = \{a_1, a_2, \dots, a_{3N}\}$  be any instance of 3-partition. We show that this instance has a 'yes'

answer if and only if there exists a solution with  $L_{\max} \leq 0$  to the following instance of Problem 1 with  $n = 4N$  jobs, consisting of  $3N$   $Ja$ -type jobs and  $N$   $Jb$ -type jobs to be defined shortly. We use  $p(k_j)$ ,  $r(k_j)$ , respectively, to denote the processing time (ready time, respectively) of job  $j$  of type  $Jk$ , where  $k = a, b$ . Specifically,

$$p(a_j) = B + 4a_j, \quad 1 \leq j \leq 3N,$$

$$p(b_j) = B, \quad 1 \leq j \leq N,$$

$$r(a_j) = 0, \quad 1 \leq j \leq 3N,$$

$$r(b_j) = (8j - 1)B, \quad 1 \leq j \leq N.$$

Also, the schedule of generalized due dates is  $3B, 6B, 7B, 8B, 11B, 14B, 15B, 16B, 19B, \dots$ , or  $d_{[11]}, d_{[13]}, d_{[14]}, d_{[21]}, d_{[22]}, \dots, d_{[N1]}, d_{[N2]}, d_{[N3]}, d_{[N4]}$ , where:

$$d_{[k1]} = 8B(k - 1) + 3B, \quad 1 \leq k \leq N,$$

$$d_{[k2]} = 8B(k - 1) + 6B, \quad 1 \leq k \leq N,$$

$$d_{[k3]} = 8B(k - 1) + 7B, \quad 1 \leq k \leq N,$$

$$d_{[k4]} = 8Bk, \quad 1 \leq k \leq N,$$

where  $d_{[ki]}$  is the due date of the  $[4(k - 1) + i]$ -th job to be completed.

(i) Suppose that a 3-partition exists, that is, we can number  $a_1, a_2, \dots, a_{3N}$  using  $a_{ik}$ ,  $i = 1, 2, 3$ ,  $k = 1, \dots, N$ , such that  $\sum_{i=1}^3 a_{ik} = B$ ,  $k = 1, \dots, N$ . Consider the following schedule  $S$  of jobs:

$$S = \{Ja_{11}, Ja_{21}, Ja_{31}, Jb_1, Ja_{12}, Ja_{22}, \dots, Ja_{1N}, Ja_{2N}, Ja_{3N}, Jb_N\}.$$

Since  $a_{ik} < \frac{1}{2}B$ , it follows that  $p(a_j) < 3B$ , and therefore that all jobs are completed on time. Thus the lateness of the schedule  $L(S) \leq 0$ .

(ii) Conversely, suppose there exists a schedule  $S^*$  such that  $L(S^*) \leq 0$ . Since  $Jb$ -type jobs are not available until  $7B$ , the first three jobs must be  $Ja$ -type. Note that  $Ja_{1r}$ ,  $Ja_{1p}$  and  $Ja_{1q}$  have processing times  $p(a_{1r})$ ,  $p(a_{1p})$ ,  $p(a_{1q})$ , respectively. Since

$$L(S^*) \leq 0,$$

$$p(a_{1r}) + p(a_{1p}) + p(a_{1q}) \leq 7B$$

$$\Rightarrow a_{1r} + a_{1p} + a_{1q} \leq B.$$

Next, note that the fourth job cannot be  $Ja$ -type job, e.g.  $Ja_{1e}$ , since  $a_{1r}, a_{1p}, a_{1q}, a_{1e} > \frac{1}{4}B \Rightarrow$

$$p(a_{1r}) + p(a_{1p}) + p(a_{1q}) + p(a_{1e}) > 8B \\ \Rightarrow L(S^*) > 0.$$

Therefore, the fourth job is  $Jb$ -type and begins processing at  $7B$  and completes processing at  $8B$ . Thus we can write  $S^*$  as

$$S^* = \{Ja_{1r}, Ja_{1p}, Ja_{1q}, Jb_1, Ja_{2r}, \dots, Ja_{Nr}, \\ Ja_{Np}, Na_{Nq}, Jb_N\},$$

with  $a_{ir} + a_{ip} + a_{iq} \leq B$  for  $1 \leq i \leq N$ . Since  $d_{[N4]} = 8BN$  and

$$\sum_{j=1}^{3N} p(a_j) + \sum_{j=1}^N p(b_j) = 8BN,$$

processing must be continuous if  $L(S^*) \leq 0$ . But if  $a_{ir} + a_{ip} + a_{iq} < B$  for some  $i$ ,  $1 \leq i \leq N$ , job  $Ja_{iq}$  will complete processing before job  $Jb_i$  is available, and processing will not be continuous. Therefore

$$L(S^*) \leq 0 \Rightarrow a_{ir} + a_{ip} + a_{iq} = B \\ \text{for } 1 \leq i \leq N$$

and a 3-partition exists.

The specific due date version of this problem is NP-complete, as shown by Lenstra, Rinnooy Kan and Brucker (1977). The next problem studied is very similar, but preemption of jobs is permitted.

**Problem 2.**  $1|gdd, pmtn, r_j|L_{\max}$ .

The specific due date version of the problem is polynomially solvable (Liu and Layland, 1973; Baker et al., 1983).

A polynomial time algorithm can be devised for Problem 2. The  $O(n \log n)$  algorithm (SPT rule based) is exactly the same as the one that solves  $1|pmtn, r_j|\sum C_j$  (refer to Baker (1974) for more details). The idea behind the algorithm is as follows. We schedule the jobs starting at the first release time. At each decision point (whenever a job is released or a job is completed) we choose to process next from among the available jobs (those whose release times have passed and for which processing is not yet complete) a job whose remaining processing time is minimal.

**Problem 3.**  $1|gdd, r_j|\sum U_j$ .

The specific due date version of this problem is NP-complete, as shown by Lenstra, Rinnooy Kan and Brucker (1977). The same complexity result holds for the generalized due date version. Recall that in proving the complexity of  $1|gdd, r_j|L_{\max}$  (Problem 1), we showed that there exists a schedule  $S^*$  for that problem with  $L(S^*) \leq 0$  if and only if there exists a 3-partition. It is easy to see that there exists a solution to Problem 3 with objective value of zero if and only if there is a schedule  $S^*$  for Problem 1 with  $L(S^*) \leq 0$ .

**Problem 4.**  $1|gdd, C_{[j]} \leq d_{[j]}|\sum w_j C_j$ .

The specific due date version of this problem is NP-complete (Lenstra, Rinnooy Kan and Brucker, 1977). We prove that this classification also holds for the generalized due date version, by transformation from 3-partition.

Given an instance of 3-partition defined by integers  $a_1, \dots, a_{3N}$  and an integer  $B$ , define an instance of Problem 4 with

$$\begin{aligned} n &= 4N - 1, \\ p(a_j) &= w_j = B + a_j, \quad j = 1, \dots, 3N \quad (Ja\text{-type jobs}), \\ w_j &= 0, \quad p(b_j) = 1, \quad j = 3N + 1, \dots, 4N - 1 \quad (Jb\text{-type jobs}), \\ d_{[j]} &= (4B + 1)\lceil j/4 \rceil, \quad j = 1, \dots, 4N - 1, \\ &\quad \text{where } \lceil x \rceil \text{ is the smallest integer greater than or equal to } x, \\ y &= 2BN(N - 1) + \sum_{1 \leq i \leq j \leq 3N} (B + a_i)(B + a_j). \end{aligned}$$

We claim that there exists a 3-partition if and only if there exists a solution to this instance of Problem 4 with objective value no greater than  $y$ .

(i) Suppose that a 3-partition exists, that is, we can number  $a_1, \dots, a_{3N}$  using  $a_{ik}$ ,  $i = 1, 2, 3$ ,  $k = 1, \dots, N$ , such that  $\sum_{i=1}^3 a_{ik} = B$ ,  $k = 1, \dots, N$ . Consider the following schedule  $S$  of jobs:

$$S = \{Ja_{11}, Ja_{21}, Ja_{31}, Jb_1, Ja_{21}, Ja_{22}, Ja_{23}, \\ Jb_2, \dots, Ja_{1N}, Ja_{2N}, Ja_{3N}\}.$$

Since  $Jb$ -type jobs have zero weight, the total weighted completion time without the insertion of  $Jb$ -type jobs would simply be

$$\sum_{1 \leq i \leq j \leq 3N} (B + a_i)(B + a_j).$$

However, the insertion of each  $Jb$ -type job delays all *subsequent*  $Ja$ -type jobs by 1 unit. Thus  $Jb_1$  delays jobs of total weight  $4B(N-1)$  by one unit,  $Jb_2$  delays jobs of total weight  $4B(N-2)$  by one unit, and so on. Thus,

$$\begin{aligned} \sum_{j=1}^n w_j C_j &= \sum_{1 \leq i \leq j \leq 3N} (B + a_i)(B + a_j) + (4B) \\ &\quad + 2(4B) + \dots + (N-1)4B = y. \end{aligned}$$

(ii) Conversely, suppose there exists a schedule  $S^*$  with objective value  $\leq y$ . Since  $d_{[4]} = 4B + 1$ , the first 4 jobs cannot all be  $Ja$ -type, or the fourth job will be late. But if two or more are  $Jb$ -type, later  $Ja$ -type jobs are delayed by at least two units, and the objective value of  $y$  cannot be achieved. Thus each group of 4 jobs consists of three  $Ja$ -type jobs followed by one  $Jb$ -type job. Let the processing times of the jobs in  $S^*$  be

$$\{p(a_{1r}), p(a_{1p}), p(a_{1q}), 1, p(a_{2r}), p(a_{2p}), p(a_{2q}), 1, p(a_{3r}), \dots\}.$$

Since  $C_{[j]} \leq d_{[j]}$ ,  $j = 1, \dots, n$ ,

$$\begin{aligned} \sum_{i=1}^t [p(a_{ir}) + p(a_{ip}) + p(a_{iq})] &\leq 4Bt, \\ t &= 1, \dots, N. \end{aligned}$$

But if

$$\begin{aligned} \sum_{i=1}^t [p(a_{ir}) + p(a_{ip}) + p(a_{iq})] &< 4Bt \\ \text{for some } t, \quad 1 \leq t \leq N-1, \end{aligned}$$

the total weight of delayed later jobs will exceed  $4B(N-t)$  and hence an objective value of  $y$  is impossible. Thus

$$\begin{aligned} \sum_{i=1}^t [p(a_{ir}) + p(a_{ip}) + p(a_{iq})] &= 4Bt \\ \text{for } 1 \leq t \leq N \end{aligned}$$

and a 3-partition exists.

**Problem 5.**  $1|gdd, \text{prec}, p_j = 1|\sum T_j$ .

While the specific due date version of this problem is NP-complete (Lenstra and Rinnooy Kan, 1978), the generalized due date version admits an efficient algorithm. It is clear that an active (consecutive) schedule is optimal. Thus, one job completes processing at each integer unit of time. Consequently any sequence satisfying the precedence constraints is optimal.

**Problem 6.**  $1|gdd, \text{prec}, p_j = 1|\sum U_j$ .

As with Problem 5, this problem is NP-complete under the specific due date definition (Lenstra and Rinnooy Kan, 1980), but efficiently solvable with generalized due dates. The analysis follows that for Problem 5.

#### 4. Multiple machine problems

The problems discussed in this section are taken from a variety of scheduling environments, but in each case  $m \geq 2$  machines are available.

**Problem 7.**  $F2|gdd|L_{\max}$ .

**Problem 8.**  $O2|gdd|L_{\max}$ .

**Problem 9.**  $J2|gdd|L_{\max}$ .

**Problem 10.**  $F2|gdd, \text{pmtn}|L_{\max}$ .

**Problem 11.**  $J2|gdd, \text{pmtn}|L_{\max}$ .

Problems 7 and 9 are NP-complete under the specific due date definition (Lenstra, Rinnooy Kan and Brucker, 1977). Again using that definition, Problem 8 is NP-complete (Lawler, Lenstra and Rinnooy Kan, 1981, 1982). Similarly, Problem 10 is NP-complete (Cho and Sahni, 1981), as is Problem 11 (Gonzalez and Sahni, 1978). Under the generalized due date definition, Problems 7–11 are all NP-complete, as a result of a transformation from 3-partition.

Given an instance of 3-partition defined by integers  $a_1, a_2, \dots, a_{3N}$  and an integer  $B$  such that  $\frac{1}{4}B < a_j < \frac{1}{2}B$ ,  $j = 1, \dots, 3N$ , and  $\sum_{j=1}^{3N} a_j =$

$NB$ , we define the following instance of Problem 7:

$$\begin{aligned} n &= 4N + 1, \\ \left\{ \begin{array}{l} p(a_j') = B/3, \\ p(a_j'') = B + a_j, \quad j = 1, \dots, 3N \text{ (two task lengths of } Ja\text{-type job } j), \end{array} \right. \\ \left\{ \begin{array}{l} p(b_j^1) = 4B, \\ p(b_j^2) = B, \quad j = 3N + 1, \dots, 4N \text{ (two task lengths of } Jb\text{-type job } j), \end{array} \right. \\ \left\{ \begin{array}{l} p(b_j^1) = 0, \\ p(b_j^2) = B, \quad j = 4N + 1, \end{array} \right. \\ d_{[1]} &= B, \\ d_{[2]} &= 5B/2, \\ d_{[3]} &= 4B, \\ d_{[4]} &= 5B, \\ d_{[j]} &= d_{[j-1]} + 5B \text{ for } 5 \leq j \leq 4N + 1. \end{aligned}$$

That is,

$$\left\{ B, \frac{5}{2}B, 4B, 5B, 6B, \frac{15}{2}B, 9B, 10B, \dots, (5N - 4)B, (5N - \frac{5}{2})B, (5N - 1)B, 5NB, (5N + 1)B \right\}.$$

(i) Suppose that a 3-partition exists, so that we can number  $a_1, \dots, a_{3N}$  as  $a_{ik}$ ,  $i = 1, 2, 3$ ,  $k = 1, \dots, N$  such that  $\sum_{i=1}^3 a_{ik} = B$ ,  $k = 1, \dots, N$ . Then the following schedule  $S$  (on both machines) has  $L(S) = 0$ .

$$S = \{ Jb_{4N+1}, Ja_{11}, Ja_{21}, Ja_{31}, Jb_{3N+1}, Ja_{12}, Ja_{22}, Ja_{32}, Jb_{3N+2}, \dots, 2b_{4N-1}, Ja_{1N}, Ja_{2N}, Ja_{3N}, Jb_{4N} \}.$$

For example,  $Jb_{4N+1}$  completes processing at  $d_{[1]} = B$  and  $Ja_{31}$  completes processing at  $d_{[4]} = 5B$ , at which time  $Jb_{3N+1}$  moves from machine 1 to machine 2. Each group of four consecutive jobs follows a similar pattern until  $Jb_{4N-1}$  completes processing at  $d_{[4N-3]}$ , jobs  $Ja_{1N}$ ,  $Ja_{2N}$  and  $Ja_{3N}$  at or before  $d_{[4N]}$ , and job  $Jb_{4N}$  at  $d_{[4N+1]}$ .

(ii) Suppose there exists a schedule  $S^*$  with  $L(S^*) \leq 0$ . Clearly the first job scheduled must be  $Jb_{4N+1}$  or the other job chosen will be late. In order for the fourth job to finish by  $d_{[4]} = 5B$  the second, third and fourth jobs must all be  $Ja$ -type. Since  $d_{[5]} = 6B$ , the fifth job must be  $Jb$ -type or it will be late. This sequence will need to be repeated throughout  $S^*$  for no job to be late. Next, note that the sum of processing times on machine 1 is  $5NB$  and on machine 2 is  $(5N + 1)B$ . Since all processing times on machine 2 are at least  $B$ ,

$d_{[4N+1]} = (5N + 1)B$  and  $L(S^*) \leq 0$ , processing must finish on machine 1 no later than  $5NB$ , and on machine 2 no later than  $(5N + 1)B$ . Consequently,  $S^*$  must provide consecutive processing on both machines if  $L(S^*) \leq 0$ . Let  $t$ ,  $1 \leq t < N$ , be the smallest integer satisfying  $\sum_{k=1}^t (a_{1k} + a_{2k} + a_{3k}) < Bt$ . Clearly if no such  $t$  exists, there is a 3-partition. But job  $Jb_{3N+t}$  does not finish processing on machine 1 until  $3Bt$ , whereas  $4Bt + \sum_{k=1}^t (a_{1k} + a_{2k} + a_{3k}) < 5Bt$ , so processing on machine 2 is not consecutive. Thus  $L(S^*) \leq 0$  implies the existence of a 3-partition, and Problem 7 is NP-complete.

Problem 9 is a generalization of Problem 7 and is thus similarly classified. Problems 8, 10 and 11 are variations under which

- (a) the same schedule  $S$  is available, and
- (b) the only schedule  $S^1$  with  $L(S^1) \leq 0$  is  $S^1 = S^*$ .

Therefore, those three problems are also NP-complete. Since we used a value of zero for  $L_{\max}$  in the proofs for Problems 7 to 11, the same results hold for the objective functions  $\sum U_j$  and  $\sum T_j$ .

**Problem 12.**  $O2 | \text{gdd, pmtn} | L_{\max}$ .

We show that the decision version of Problem 12 is NP-complete in the strong sense by transformation from 3-partition. Given an instance of 3-partition defined by integers  $a_1, a_2, \dots, a_{3N}$  and an integer  $B$  such that  $B/4 < a_j < B/2$ ,  $j = 1, 2, \dots, 3N$  and  $\sum_{j=1}^{3N} a_j = NB$ , we define the following instance of the decision version of Problem 12:

$$\begin{aligned} n &= 3N, \\ \left\{ \begin{array}{l} p_j^1 = 3a_j, \\ p_j^2 = 2B - 3a_j, \quad j = 1, \dots, 3N \text{ (two task lengths of job } j), \end{array} \right. \\ d_{[k]} &= 3B \lceil k/3 \rceil, \quad 1 \leq k \leq 3N; \\ &\quad (\text{i.e., } d_{[1]} = 3B, d_{[2]} = 3B, d_{[3]} = 3B, d_{[4]} = 6B, d_{[5]} = 6B, d_{[6]} = 6B, \dots, d_{[3N-2]} = d_{[3N-1]} = d_{[3N]} = 3NB), \\ y &= 0. \end{aligned}$$

We show that there is a schedule  $S$  for the decision problem given above with maximum lateness  $L(S) \leq y$  if and only if there exists a 3-partition.

(i) Suppose there is a 3-partition  $\{A_1, A_2, \dots, A_N\}$ . The sum of the processing times on the first

machine  $M_1$  (hence on  $M_2$ ) of the 3 jobs corresponding to the set  $A_1$  is  $3B$ . The polynomial time algorithm of Gonzalez and Sahni (1976), later improved by Gonzalez (1979), for  $O|\text{pmtn}|C_{\max}$  can be used to find a feasible preemptive schedule  $S$  for the 3 jobs that correspond to the set  $A_1$  in the interval  $[0, 3B]$ . Similarly the jobs that correspond to the set  $A_i$ ,  $i = 1, 2, \dots, N$ , can be scheduled in the interval  $[3B(i-1), 3Bi]$  respectively. Hence the schedule  $S$  obtained has maximum lateness  $L(S) = 0$ . It should be noted that for the problem  $O2|\text{pmtn}|C_{\max}$ , preemption cannot reduce cost. Thus the nonpreemptive algorithm for  $O2||C_{\max}$  (Gonzalez and Sahni, 1976) can also be used to obtain the feasible schedule in the intervals described.

(ii) Supposing there is a schedule  $S_0$  such that  $L(S_0) \leq 0$ , we show that there exists a desired 3-partition  $\{A_1, A_2, \dots, A_N\}$ . Since  $L(S_0) \leq 0$  and  $\frac{1}{4}B < a_j < \frac{1}{2}B$ , the number of jobs (say, set  $J_1$  consists of these jobs) processed before time  $3B$  is no more than 3 and no less than 3. Moreover, the sum of the processing times of jobs  $J_1$  on  $M_1$  (hence on  $M_2$ ) has to be  $3B$ . Similarly, it is easy to see that in  $S_0$ , three jobs of total processing time  $3B$  on each machine are scheduled in each interval  $[3B(i-1), 3Bi]$ ,  $i = 1, 2, \dots, N$ . Hence there exists a 3-partition.

Note that the specific due date version of problem 12 can be solved by linear programming (Cho and Sahni, 1981). Their formulation is similar to the one given in Lawler and Labetoulle (1978) for an environment of unrelated parallel machines. The recent development of a polynomial time algorithm for linear programming (Khachian, 1979) means that this problem is solvable in polynomial time. Lawler, Lenstra and Rinnooy Kan (1981) provide another polynomial time algorithm for the special case  $m = 2$ .

**Problem 13.**  $O2|\text{gdd}, \text{pmtn}|\Sigma U_j$ .

**Problem 14.**  $O2|\text{gdd}, \text{pmtn}|\Sigma T_j$ .

Since we used a value of zero for  $L_{\max}$  in the proof of Problem 12, the same result holds for both of the above problems.

**Problem 15.**  $P|\text{gdd}, \text{pmtn}|L_{\max}$ .

This problem can be solved by reformulating any instance to an equivalent specific due date

instance. Then solution is possible by the efficient algorithm of Horn (1974). Let us first describe the algorithm for our problem.

**Algorithm.** (1) Number the jobs so that  $p_1 \leq p_2 \leq \dots \leq p_n$ . (Recall that  $d_{[1]} \leq d_{[2]} \leq \dots \leq d_{[n]}$  (generalized due dates).)

(2) Consider the following specific due date problem  $P|\text{pmtn}|L_{\max}$ : job 1 has processing time  $p_1$  with due date  $d_{[1]}$ , job 2 has processing time  $p_2$  with due date  $d_{[2]}$ , ..., job  $n$  has processing time  $p_n$  with due date  $d_{[n]}$ .

(3) Solve the specific due date problem above by the efficient algorithm of Horn (1974), and use the solution and associated schedule for our problem  $P|\text{gdd}, \text{pmtn}|L_{\max}$ .

We shall now demonstrate that the above algorithm is optimal.

Let  $S^*$  be an optimal schedule for an instance of  $P|\text{gdd}, \text{pmtn}|L_{\max}$  with  $L_{\max} = L^*$ , where  $d_{[1]} \leq d_{[2]} \leq \dots \leq d_{[n]}$ . Note that in  $S^*$ , we have  $C_{[j]} \leq d_{[j]} + L^*$ ,  $j = 1, 2, \dots, n$ . Let job  $j$  be the  $j$ -th job to finish and  $C_{[j]}$  be its completion time. Moreover  $C_{[1]} \leq C_{[2]} \leq \dots \leq C_{[n]}$ . Now we show that there exists an optimal schedule for any instance of  $P|\text{gdd}, \text{pmtn}|L_{\max}$  such that  $p_{[1]} \leq p_{[2]} \leq \dots \leq p_{[n]}$ , where  $p_{[i]}$  is the processing time of the  $i$ -th job completed. Suppose  $S^*$  does not satisfy this condition. Then there exist two consecutive jobs (the  $k$ -th and  $(k+1)$ -st jobs in  $S^*$ ) such that  $p_{[k+1]} < p_{[k]}$ . Note that  $C_{[k]} \leq C_{[k+1]}$ . We show below that by interchanging portions of these jobs in  $S^*$ , we can force the  $(k+1)$ -st job to finish no later than  $C_{[k]}$  and the  $k$ -th job to finish at  $C_{[k+1]}$ . Hence this operation does not increase the value of  $L^*$ . After a finite number of such interchanges in  $S^*$ , we can get another schedule  $S_0^*$ , whose lateness is no more than  $L^*$ , in which  $p_{[1]}^0 \leq p_{[2]}^0 \leq \dots \leq p_{[n]}^0$ , where  $p_{[i]}^0$  is the processing time of the  $i$ -th job scheduled in  $S_0^*$ . Since the value of  $L^*$  does not increase as a result of the interchanges,  $S_0^*$  is also optimal. Note that schedule  $S_0^*$  has the due date specified to the jobs, i.e., the shortest job finishes by the due date  $d_{[1]}$  ( $C_{[1]}^0 \leq d_{[1]} + L^*$ ), the second shortest job finishes by the due date  $d_{[2]}$  ( $C_{[2]}^0 \leq d_{[2]} + L^*$ ), and so on, until finally the longest job finishes by the last due date  $d_{[n]}$  ( $C_{[n]}^0 \leq d_{[n]} + L^*$ ).

Table 1  
Complexity status of generalized due date scheduling

| Problem (notation for specific due date)                                 | Generalized due date version             | Specific due date version                                          |
|--------------------------------------------------------------------------|------------------------------------------|--------------------------------------------------------------------|
| $1   L_{\max}$                                                           | Polynomially solvable, Hall (1986)       | Polynomially solvable, Jackson (1955)                              |
| $1   \sum T_j$                                                           | Polynomially solvable, Hall (1986)       | NP-complete, Du and Leung (1990)                                   |
| $1   r_j   \sum T_j$                                                     | NP-complete, Hall (1986)                 | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $1   \sum U_j$                                                           | Polynomially solvable, Hall (1986)       | Polynomially solvable, Moore (1968)                                |
| $1   \sum w_j U_j$                                                       | NP-complete, Hall (1986)                 | NP-complete, Karp (1972)                                           |
| $1   r_j   L_{\max}$                                                     | NP-complete (Problem 1)                  | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $1   \text{pmtn}, r_j   L_{\max}$                                        | Polynomially solvable (Problem 2)        | Polynomially solvable, Liu and Layland (1973), Baker et al. (1983) |
| $1   r_j   \sum U_j$                                                     | NP-complete (Problem 3)                  | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $1   C_{[j]} \leq d_{[j]}   \sum w_j C_j$                                | NP-complete (Problem 4)                  | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $1   \text{prec}, p_j = 1   \sum T_j$                                    | Polynomially solvable (Problem 5)        | NP-complete, Lenstra and Rinnooy Kan (1978)                        |
| $1   \text{prec}, p_j = 1   \sum U_j$                                    | Polynomially solvable (Problem 6)        | NP-complete, Lenstra and Rinnooy Kan (1980)                        |
| $1   \sum w_j T_j$                                                       | NP-complete, Sriskandarajah (1990)       | NP-complete, Lawler (1977)                                         |
| $1   \text{prec}   L_{\max}$                                             | NP-complete, Sriskandarajah (1990)       | Polynomially solvable, Lawler (1973)                               |
| $1   \text{pmtn}, \text{prec}, r_j   L_{\max}$                           | NP-complete, Sriskandarajah (1990)       | Polynomially solvable, Blazewicz (1976), Baker et al. (1983)       |
| $P2   L_{\max}$                                                          | NP-complete, Hall (1986)                 | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $P2   \sum U_j$                                                          | NP-complete, Hall (1986)                 | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $F2   \gamma, \gamma \in \{L_{\max}, \sum U_j, \sum T_j\}$               | NP-complete (Problem 7)                  | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $O2   \gamma, \gamma \in \{L_{\max}, \sum U_j, \sum T_j\}$               | NP-complete (Problem 8)                  | NP-complete, Lawler, Lenstra and Rinnooy Kan (1981, 1982)          |
| $J2   \gamma, \gamma \in \{L_{\max}, \sum U_j, \sum T_j\}$               | NP-complete (Problem 9)                  | NP-complete, Lenstra, Rinnooy Kan and Brucker (1977)               |
| $F2   \text{pmtn}   \gamma, \gamma \in \{L_{\max}, \sum U_j, \sum T_j\}$ | NP-complete (Problem 10)                 | NP-complete, Cho and Sahni (1981)                                  |
| $J2   \text{pmtn}   \gamma, \gamma \in \{L_{\max}, \sum U_j, \sum T_j\}$ | NP-complete (Problem 11)                 | NP-complete, Gonzalez and Sahni (1978)                             |
| $O   \text{pmtn}   L_{\max}$                                             | NP-complete even if $m = 2$ (Problem 12) | Polynomially solvable, Cho and Sahni (1981)                        |
| $O2   \text{pmtn}   \sum U_j$                                            | NP-complete (Problem 13)                 | NP-complete, Lawler, Lenstra and Rinnooy Kan (1981)                |
| $O2   \text{pmtn}   \sum T_j$                                            | NP-complete (Problem 14)                 | NP-complete, Du and Leung (1990)                                   |
| $P   \text{pmtn}   L_{\max}$                                             | Polynomially solvable (Problem 15)       | Polynomially solvable, Horn (1974)                                 |

Table 2

Open problems in generalized due date scheduling

| Problem<br>(notation for specific due date) | Generalized due<br>date version | Specific due date version                                                                                                                  |
|---------------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| $1 \text{prec}, p_j=1 \sum w_j T_j$         | Open                            | NP-complete even if $w_j=1$<br>and precedence relation is chain,<br>Leung and Young (1989),<br>Lenstra and Rinnooy Kan (1978, 1980, 1984). |
| $1 \text{prec}, p_j=1 \sum w_j U_j$         | Open                            | NP-complete even if $w_j=1$ and<br>precedence relation is chain,<br>Lenstra and Rinnooy Kan (1980)                                         |
| $P \text{pmtn} \sum T_j$                    | Open                            | NP-complete, Du and Leung (1990)                                                                                                           |
| $P \text{pmtn} \sum U_j$                    | Open                            | NP-complete, Lawler (1982)                                                                                                                 |

### Interchanging portions of the $k$ -th and $(k+1)$ -st job in $S^*$

Let  $R_j^i$  denote the total time during which the  $j$ -th job to finish is being processed and the other job is not, in the time interval  $[0, C_{[k]}]$  if  $i=1$ , or  $[C_{[k]}, C_{[k+1]}]$  if  $i=2$ , for  $j=k, k+1$ . Since  $p_{[k+1]} < p_{[k]}$ , it follows immediately that  $R_{k+1}^1 + R_{k+1}^2 < R_k^1 + R_k^2 \Rightarrow R_{k+1}^1 + R_{k+1}^2 < R_k^1$ , since  $R_k^2 = 0, \Rightarrow R_{k+1}^2 < R_k^1$ . Now it is possible to interchange the  $R_{k+1}^2$  portion of the  $(k+1)$ -st job with the same processing time from  $R_k^1$  of the  $k$ -th job. Following this interchange, the finish times of the  $k$ -th and  $(k+1)$ -st jobs are no later than  $C_{[k+1]}$  and  $C_{[k]}$  respectively.

## 5. Summary and conclusions

We have studied the complexity of a wide variety of generalized due date scheduling problems. Most of the better known machine scheduling problems have now had their complexity resolved (Table 1), although some open problems remain (Table 2). Interest in generalized due date scheduling problems is likely to increase in the future, as increased flexibility becomes important in manufacturing environments. Our early work on this problem class will therefore provide the starting point for further research studies.

## Acknowledgements

The authors wish to thank two anonymous referees, whose detailed comments on an earlier draft improved the paper.

This research was supported by NSERC Grants A4619, OGP 0104900 MRCO and FCAR Grant 89-EQ-4144.

## References

- Baker, K.R. (1974), *Introduction to Sequencing and Scheduling*, Wiley, New York.
- Baker, K.R., Lawler, E.L., Lenstra, J.K., and Rinnooy Kan, A.H.G. (1983), "Preemptive scheduling of a single machine to minimize maximum cost subject to release dates and precedence constraints", *Operations Research* 31, 381–386.
- Blazewicz, J. (1976), "Scheduling dependent tasks with different arrival times to meet deadlines", in: *Modelling and Performance Evaluation of Computer Systems*, E. Gelenbe and H. Beilner (eds.), North-Holland, Amsterdam, 57–65.
- Browne, J., Dubois, D., Rathmill, K., Sethi, S.P., and Steckel, K. (1984), "Classification of flexible manufacturing systems", *The FMS Magazine*, 114–117.
- Cho, Y., and Sahni, S. (1981), "Preemptive scheduling of independent jobs with release and due times on open, flow and job shops", *Operations Research* 29/3, 511–522.
- Conway, R.W., Maxwell, W.L., and Miller, L.W. (1967), *Theory of Scheduling*, Addison-Wesley, Reading, MA.
- Du, J., and Leung, J.Y-T. (1990), "Minimizing total tardiness on one machine is NP-hard", *Mathematics of Operations Research* 15, 483–495.
- Garey, M.R., and Johnson, D.S. (1979), *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, New York.

- Gonzalez, T. (1979). "A note on open shop preemptive schedules", *IEEE Transactions on Computers* C-28, 782–786.
- Gonzalez, T., and Sahni, S. (1976), "Open shop scheduling to minimize finish time", *Journal of Association for Computing Machinery* 23, 665–679.
- Gonzalez, T., and Sahni, S. (1978), "Flowshop and jobshop schedules: complexity and approximation", *Operations Research* 26, 36–52.
- Graham, R.L., Lawler, E.L., Lenstra, J.K., and Rinnooy Kan, A.H.G. (1979), "Optimization and approximation in deterministic sequencing and scheduling: a survey", *Annals of Discrete Mathematics* 5, 287–326.
- Hall, N.G. (1986), "Scheduling problems with generalized due dates", *IIE Transactions* 18, 220–222.
- Horn, W.A. (1974), "Some simple scheduling algorithms", *Naval Research Logistics Quarterly* 21, 177–185.
- Jackson, J.R. (1955), "Scheduling a production line to minimize maximum tardiness", *Research Report* 43, Management Science Research Project, University of California, Los Angeles.
- Karp, R.M. (1972), "Reducibility among combinatorial problems", in: *Complexity of Computer Computations*, R.E. Miller and J.W. Thatcher (eds.), Plenum Press, New York, 85–103.
- Khachian, L.G. (1979), "A polynomial algorithm for linear programming", *Doklady Akademii Nauk. SSSR* 244, 1093–1096.
- Lawler, E.L. (1973), "Optimal sequencing of a single machine subject to precedence constraints", *Management Science* 19, 544–546.
- Lawler, E.L. (1977), "A pseudopolynomial algorithm for sequencing jobs to minimize total tardiness", *Annals of Discrete Mathematics* 1, 331–342.
- Lawler, E.L. (1982), "Recent results in the theory of machine scheduling", in: *Mathematical Programming: The State of the Art*, A. Bachem, M. Grotschel and B. Korte (eds.) Springer, Berlin, 202–234.
- Lawler, E.L., and Labetoulle, J. (1978), "On preemptive scheduling of unrelated parallel processors by linear programming", *Journal of the Association for Computing Machinery* 25, 612–619.
- Lawler, E.L., Lenstra, J.K., and Rinnooy Kan, A.H.G. (1981), "Minimizing maximum lateness in a two-machine openshop", *Mathematics of Operations Research* 6, 153–158.
- Lawler, E.L., Lenstra, J.K., and Rinnooy Kan, A.H.G. (1982), "Erratum", *Mathematics of Operations Research* 7, 635.
- Lenstra, J.K., Rinnooy Kan, A.H.G., and Brucker, P. (1977), "Complexity of machine scheduling problems", *Annals of Discrete Mathematics* 1, 343–362.
- Lenstra, J.K., and Rinnooy Kan, A.H.G. (1978), "Complexity of scheduling under precedence constraints", *Operations Research* 26, 22–35.
- Lenstra, J.K., and Rinnooy Kan, A.H.G. (1980), "Complexity results for scheduling chains on a single machine", *European Journal of Operational Research* 4, 270–275.
- Lenstra, J.K., and Rinnooy Kan, A.H.G. (1984), "Two open problems in precedence constrained scheduling", *Annals of Discrete Mathematics* 23, 509–522.
- Leung, J.Y-T., and Young, G.H. (1989), "Minimizing total tardiness on a single machine with precedence constraints", *Technical Report*, Computer Science Program, University of Texas at Dallas, Richardson, TX 75083.
- Liu, C.L., and Layland, J.W. (1973), "Scheduling algorithms for multiprogramming in a hard-real-time environment", *Journal of the Association for Computing Machinery* 20/1, 46–61.
- Moore, J.M. (1968), "An  $n$  job, one machine sequencing algorithm for minimizing the number of late jobs", *Management Science* 15, 102–109.
- Papadimitriou, C.H., and Steiglitz, (1982), *Combinatorial Optimization*, Prentice-Hall, Englewood Cliffs, NJ.
- Sriskandarajah, C. (1990), "A note on the generalized due dates scheduling problem", *Naval Research Logistics* 37, 587–597.
- Stecke, K.E., and Solbert, J.J. (1981), "Loading and control policies for a flexible manufacturing system", *International Journal of Production Research* 19, 481–490.